CORTEX XSIAM · OFFICIAL MARKETPLACE CONTENT PACK

KOI

demisto/content · Packs/Koi · v1.2.3

support: xsoar author: Cortex XSOAR category: Endpoint from XSOAR 6.10.0

Visibility over the software your workforce installs itself — browser and IDE extensions, MCP servers, code packages and OS software — across every endpoint KOI sees.

13	5	1	0
commands	context prefixes	integration	playbooks or rules

This is the Marketplace pack. A different, in-house KOI pack (v1.3.0, 26 commands) also exists and cannot coexist with it — see the comparison slide.

WHAT IT COVERS

The self-installed software surface

7 of the 13 commands declare a marketplace filter (22 values); only koi-inventory-list declares a platform filter (45). Filter vocabularies, not measured reach. Tokens in the five family cards are verbatim from that marketplace enum; the view card is the API's list.

3 Browser extensions

`chrome_web_store` · `edge_add_ons`
`firefox_add_ons`

7 IDE & editor extensions

`vscode` · `visual_studio` · `jetbrains`
`cursor` · `windsurf`
`open_vsx_registry` · `notepad++`

3 Agentic AI & MCP

`github_mcp_registry`
`claude_desktop_extensions`
`hugging_face`

3 Code & container packages

`npm` · `pypi` · `docker`

6 OS software & package managers

`windows` · `mac` · `linux` · `homebrew`
`chocolatey` · `office_add_ins`

view — the API's 9 accepted values

`agentic_ai` · `ai_models` · `code_packages`
`extensions` · `os_packages` · `software`
`all_items` · `mcp_servers` · `repositories`

grey = in the pack's view enum · amber = API-only, absent from it [LIVE]

Filter and sort on top of that: risk_level (low, medium, high, critical, pending), installation_method (marketplace, manual, built_in, side_loaded), publisher, first-seen date and a query-builder search.

Family tokens: the marketplace argument enum in marketplace-pack.json; the headings are an editorial grouping, not a pack construct. The view card lists the API's accepted set, three values of which the pack's own enum omits: VERIFIED_FACTS §5.1 [LIVE].

CONTENTS

What actually ships

A — **One integration**

KOI — id KOI, category Endpoint.

B — **13 commands**

67 arguments, 131 output declarations over 68 distinct context paths.

C — **An event collector — XSIAM / platform only**

isfetchevents true, xsoar false — Alerts and Audit. Disabled on XSOAR.

— — **No playbooks**

Nothing orchestrates the commands. You supply the workflow.

— — **No parsing or modeling rules**

None ships in the pack, and no XDM field is populated in koi_koi_raw.

— — **No dashboard, no layouts**

Integration only. Everything else is yours to build.

Build facts

Pack version
`1.2.3`

Integration id
`KOI`

From version
`6.10.0`

Docker image
`demisto/fastapi:0.125.0.10158186`

Marketplaces
`xsoar, marketplacev2, platform`

Support
`xsoar`

Config parameters
`9`

Required at setup — Connect
`url, api_key`

event_types_to_fetch is required too, but it is a Collect parameter hidden on XSOAR — there it is not a setup input at all.

4 Connect and 5 Collect parameters; every Collect one is hidden on XSOAR.

All values, including required and hidden: [xsoar], generated at build time from marketplace-pack.json [YAML]; isfetchevents XSOAR override, YAML lines 990–991. "Integration only" — no playbooks, rules or dashboard — is an inspection of the pack directory, not a tenant observation. That no XDM field is populated in koi_koi_raw is [LIVE]: VERIFIED_FACTS §3.2.

DATA MODEL

Five context prefixes, and an item-centric shape

KOI.Event

4

distinct paths

koi-get-events

Koi.Policy

9

distinct paths

policy list + status update

Koi.Allowlist

9

distinct paths

koi-allowlist-get

Koi.Blocklist

9

distinct paths

koi-blocklist-get

Koi.Inventory

37

27 item + 10 nested Endpoint

list · item-get · search · item-endpoints-list

4 + 9 + 9 + 9 + 37 = 68 distinct context paths — the Koi.Inventory card includes the 10 nested Koi.Inventory.Endpoint.* paths.

Item-centric: an endpoint is reached only through an item

koi-inventory-list

or -search
find items

→

Koi.Inventory.*

27 item fields
risk, publisher,
governance

→

koi-inventory-item-endpoints-list

item_id + marketplace
+ version

→

Koi.Inventory.Endpoint.*

10 nested fields — hostname, os, serial, path, last_logged_on_user

Two things that bite

There is no Koi.Device.* prefix. Endpoints exist only under Koi.Inventory.Endpoint.*, reached from an item — you cannot ask this pack "what is on host X" directly. **The casing is inconsistent:** KOI.Event for events, Koi.* for everything else. Nothing in the pack says why. Preserve it exactly as declared rather than normalising it — DT paths are case-sensitive.

Every number on this slide is computed at build time from marketplace-pack.json (131 declarations, 68 distinct paths, 36 of them declared by more than one command); the build fails if the five cards stop summing to 68. Absence of Koi.Device.: VERIFIED_FACTS §4.*

COMMAND SURFACE

13 commands, all of them

1 Events

`get-events`

5 args · 4 outputs

2 Policy

`policy-list`
`policy-status-update`

5 args · 18 outputs

3 Allowlist

`allowlist-get`
`allowlist-items-remove`
`allowlist-items-add`

10 args · 9 outputs

3 Blocklist

`blocklist-get`
`blocklist-items-remove`
`blocklist-items-add`

10 args · 9 outputs

4 Inventory

`inventory-list`
`inventory-item-get`
`inventory-search`
`inventory-item-endpoints-list`

37 args · 91 outputs

all prefixed koi- · amber = execution: true · 5 commands change tenant state, but only these 2 carry the flag · non-mutating is not the same as freely repeatable — see get-events below

8 of 13 non-mutating

The other 5 change tenant state. The pack flags only 2:
`allowlist-items-remove`
`blocklist-items-remove`
allowlist-items-add, blocklist-items-add, policy-status-update mutate with no flag at all.
Only 7 of the 8 are freely repeatable: get-events may duplicate events and disrupt the fetch.

4 return no context

All 4 add/remove commands declare no outputs at all. A playbook cannot branch on their result.

63 redundant declarations

131 declarations, 68 distinct paths. 36 paths are declared by more than one command — the three inventory item commands and both policy commands overwrite each other's context.

Groups, names, counts and the execution flag are all derived at build time from marketplace-pack.json. Non-mutating / state-changing split and the get-events caveat: VERIFIED_FACTS §1.1 and §4.1. Overwrite: §4.

Marketplace KOI pack v1.2.3 · demisto/content / Packs/Koi · integration only, 13 commands · not the custom v1.3.0 pack

EVENT COLLECTION — XSIAM / PLATFORM ONLY

One dataset, two incompatible schemas

KOI API

https://api.prod.koi.security/
Alerts + audit logs

→

Integration

isfetchevents true
xsoar false — XSIAM only
every 1 min, max 5000/type

→

koi_koi_raw

_vendor koi
_product koi

→

XQL only

no XDM fields
query raw fields

source_log_type = Audit

19,842

events in 30 d · Flat, KOI-native

```
type · action · category · object_name · object_type
hostname · item_version
type values seen:
extensions 16,579 · devices 2,971 · remediation 244 · policies 32
approval_requests 8 · guardrails 6 · notifications 2
```

source_log_type = Alerts

314

events in 30 d · OCSF

```
class_uid 2007 (Application Security Posture Finding)
type_uid 200701 · is_alert true
severity_id · confidence_id · risk_level_id · status_id
resources / observables / metadata are JSON STRINGS —
XQL must json_extract them or it silently returns nothing
```

! **Nothing is normalised.** No parsing rule and no modeling rule ships in the pack, and no XDM field is populated at all — every Audit field is null on Alert rows and vice versa. **alert_type is never populated**: any query or playbook keyed on it matches nothing.

Dataset name, schema split and the JSON-string caveat: VERIFIED_FACTS §3 [LIVE, 30-day window to 20 Jul 2026, tenant api-ayman.xdr.eu]. The event counts are that one window on that one tenant and keep growing — a snapshot, not a figure to reproduce. Absence of rules: pack inspection. Fetch defaults: marketplace-pack.json [YAML]; event types offered: Alerts,Audit.

Marketplace KOI pack v1.2.3 · demisto/content / Packs/Koi · integration only, 13 commands · not the custom v1.3.0 pack

BEFORE YOU BUILD ON IT

Two defects in the pack, and four things that are not defects

KINDS ■ Defect in the shipped pack (2) ■ Documentation gap (1) ■ API observation (2) ■ This tenant's configuration (1)

1 DEFECT IN THE SHIPPED PACK

The view dropdown is incomplete

The API accepts nine values; the YAML offers 6. Missing: all_items, mcp_servers, repositories. All twelve were probed on both instances: mcp_servers holds real data (21 items on KOI_PAET, 42 on KOI_PLTS) but is never offered, and all_items is accepted yet returned zero rows on both.

2 DEFECT IN THE SHIPPED PACK

The shipped example is broken

The shipped command_examples.txt uses view=browser_extensions — a value in neither the YAML enum nor the API's accepted set. The live API answers 400, so anyone copying the example gets an error. ide_extensions and packages also 400, but are not in it.

3 DOCUMENTATION GAP

inventory-search needs a filter shape

The API wants {"combinator":"and","rules":[...]}; the pack never shows one. Send {} and it answers 400, naming the bad keys. Omitting the filter is blocked inside Koi.py before any call — read from the source, not executed. A missing example, not a bad error.

4 API OBSERVATION · RESPONSE SHAPE

Allow/blocklist carry no total

/policies/allowlist and /policies/blocklist return only an items array — no total_count, unlike /policies and /inventory. Read items.length instead. An empty array is the correct answer for an empty list, not an error.

5 API OBSERVATION · BEHAVIOUR

Auth failure is unambiguous — 401

An invalid bearer token returns HTTP 401 Unauthorized, verified with a deliberately bad key. No 403 was ever observed from here, so "403 means blocked egress" stays unverified and must not be stated as fact.

6 THIS TENANT'S CONFIGURATION

Two collectors, one dataset

Not a pack fault — how this tenant happens to be configured. Both instances were set to fetch Alerts and Audit into the same koi_koi_raw, and the pack ships no field identifying which instance produced a row.

None of these is a blocker. Each has a workaround: type the view value by hand, ignore the shipped example, paste the filter shape, read items.length instead of total_count, prefer omitting view to using all_items, and tag rows by instance yourself if you run more than one.

Colour = kind, as in the legend. Cards 1–5: VERIFIED_FACTS §5 [LIVE against the KOI API, 20 Jul 2026, instances KOI_PAET and KOI_PLTS], except the Koi.py guard in card 3, read from the integration source rather than executed. Card 6: §2, this tenant's instance configuration. Only cards 1 and 2 are faults in the pack as published.

Marketplace KOI pack v1.2.3 · demisto/content / Packs/Koi · integration only, 13 commands · not the custom v1.3.0 pack

DO NOT CONFUSE THEM

There are two KOI packs, and they collide

This pack — Marketplace

`demisto/content · Packs/Koi · v1.2.3`

- 13 commands
- Integration only
- No playbooks
- No parsing or modeling rules
- No dashboard
- 5 context prefixes, item-centric
- Installed from the Marketplace

The other pack — custom

`in-house build · v1.3.0 · not on the Marketplace`

- 26 commands (a strict superset)
- Integration + 10 playbooks
- Parsing and modeling rules
- An alerts dashboard
- Adds device-centric commands and context
- Adds Koidex catalog, findings, approvals, remediations
- Delivered as a pack zip or source

They cannot be installed on the same tenant

Both are named KOI. Both carry integration id KOI. Both are category Endpoint, both are authored by "Cortex XSOAR", both target xsoar/marketplacev2/platform, both use koi-* command names. **One silently overwrites the other.**

Anything written against the custom pack's extra 13 commands — devices-list, koidex-, findings-list, users-list, remediations-list, runtime-policies and the rest — does not work here, and neither does any Koi.Device.* path.*

Left column: marketplace-pack.json + VERIFIED_FACTS. Right column verified by reading the custom pack checkout at ../KOI on 20 Jul 2026 (pack_metadata.json v1.3.0, same three marketplaces; Integrations/Koi/Koi.yml id KOI, 26 koi- commands; 10 playbook files; ModelingRules/, ParsingRules/, XSIAMDashboards/).*

SCOPE

What you would build on top of it

The pack gives you a clean API surface, and on XSIAM a working event collector. It does not give you content. Sized honestly, that is four pieces of work.

1

Normalisation

Parsing and modeling rules for koi_koi_raw. Two schemas in one dataset, keyed on source_log_type, with JSON-string fields on the Alert side that need extracting before anything can be mapped to XDM.

2

Detection & triage content

Correlation rules over the raw fields, and whatever decides which items matter. Nothing keys on alert_type — it is never populated.

3

Orchestration

Playbooks that chain the 13 commands. Watch the shared context prefixes: the 3 inventory commands that share the item paths overwrite each other, and the 4 add/remove commands return nothing to branch on.

4

Visualisation & reporting

Dashboards and widgets. Everything must be written against raw fields until the modeling rules above exist.

This is scope, not criticism

The Marketplace pack does the part that is hard to do yourself — an authenticated, paginated, versioned client over the KOI API, plus a supported event collector on XSIAM. The content layer above it is the part that is specific to your estate, and it is the part you would want to own anyway.

Editorial: this slide is an inference drawn from the gaps recorded in VERIFIED_FACTS §3.2, §4 and §5. It states no new fact about the pack.

PROVENANCE

What was checked, and what was not

Verified

on the tenant or against the KOI API, 20 Jul 2026

- Which pack is installed — the Marketplace pack, brand KOI
- The dataset name koi_koi_raw — measured, not inferred
- 20,156 events and 80 distinct hostnames over 30 days — a snapshot that drifts
- The Audit / Alerts schema split and the OCSF class
- That no XDM field is populated anywhere
- Fetch events enabled on both instances — XSIAM only
- The API behind 8 of the 13 commands, exercised without changing tenant state — 7 of those are freely repeatable, get-events is not
- The 5 state-changing commands were deliberately NOT run: allowlist-items-remove, allowlist-items-add, blocklist-items-remove, blocklist-items-add, policy-status-update
- An invalid bearer token returns HTTP 401 (no 403 was ever observed)
- The two pack defects on the sharp-edges slide — the four other cards there are a documentation gap, two API observations and this tenant's configuration

Not verified

stated as unverified, never asserted

- Not one of the 13 commands was executed through XSIAM — no war-room output and no context mapping was observed
- POST /investigations/search returns a 303 to /#/404, so demisto-sdk run cannot reach this tenant at all
- POST /incident returns 200 with an empty body and creates nothing
- An API-key user's playground is a malformed stub — every command there panics, including the built-in !Print
- So: XSOAR-side context mapping and human-readable output are asserted from the YAML and Koi.py, not observed
- The five state-changing commands, whose real behaviour is untested by design
- To close it: run the sweep from the XSIAM war room and compare against evidence/command-sweep.json

How this deck is built: every command, argument, enum, context path and configuration value on these slides is read at build time from reference/marketplace-pack.json, derived mechanically from demisto/content Packs/Koi/Integrations/Koi/Koi.yml (md5 5497cdddedeb0c0d7d0b371aa075a64c, checked against master 2026-07-20). Nothing about the command surface is hand-typed. The endpoint and auth details come from Koi.py, not the YAML.

VERIFIED_FACTS §1.1, §3, §4.1, §5 and §6.

IN ONE LINE

A supported client, and a collector on XSIAM. The content layer is yours.

1

Install it if

you want 13 supported commands over the KOI inventory, policy and allow/block lists — and, on XSIAM, Alerts and Audit into koi_koi_raw with no code to maintain.

2

Budget for

normalisation and triage content. There are no rules, no playbooks and no dashboard in this pack.

3

Check first

that the other KOI pack is not installed. Same name, same integration id — one overwrites the other.

KOI · Marketplace pack v1.2.3 · demisto/content / Packs/Koi/Integrations/Koi/Koi.yml · 13 commands · integration only

Marketplace KOI pack v1.2.3 · demisto/content / Packs/Koi · integration only, 13 commands · not the custom v1.3.0 pack